OmniMemory Project Documentation

Documentation snapshot: October 23, 2026 **Repository:** `D:\aimemory` **Application:** OmniMemory 0.1.0 **Primary platform:** Windows x64 desktop application, with cross-platform local development support

This document is the consolidated project guide. It explains what OmniMemory does, how the repository is organized, how data and security work, how to run each interface, how to build the desktop package, and what remains unvalidated.

The source documentation remains split by concern in `docs/`. This document is intentionally broader and is suitable for onboarding a maintainer, operator, or agent who needs the whole system picture.

Contents

1 Product summary

2 Capabilities

3 Repository map

4 Architecture

5 Memory model

6 Projects, tasks, and handoffs

7 Authentication and authorization

8 First-run desktop onboarding

9 REST API

10 Browser PWA

11 Local MCP integration

12 SQLite database and migrations

13 Configuration

14 Desktop runtime and packaging

15 Public marketing website

16 Security model

17 Installation and operating procedures

18 Testing and verification

19 Current deliverables

20 Limitations and deferred work

21 Maintainer checklist

Product summary

OmniMemory is a local-first memory and coordination system for authorized AI agents. It keeps the authoritative store in SQLite on the local machine and exposes several controlled interfaces over the same service layer:

1. An authenticated loopback-only REST API.
1. A responsive browser PWA served by that API.
1. A local stdio MCP server for coding agents.
1. A Windows Electron desktop shell that owns and supervises the backend process.
1. A separate static public marketing website.

The system is intentionally provider-independent. It does not call external AI APIs, does not sync to a cloud service, does not expose a LAN listener, and does not provide remote MCP transport.

The authoritative design principle is:

> *Memory, authorization, validation, persistence, and history live in shared local services. REST,
PWA, MCP, and Electron are adapters around those services rather than separate implementations.*

Capabilities

Memory and knowledge

1. Create and manage projects.
1. Store project-scoped memories with typed records.
1. Store provenance for every memory revision.
1. Search current records with SQLite FTS5 lexical search.
1. Keep immutable revision history.
1. Reject stale updates using optimistic version checks.
1. Record memory-domain activity and commit sequencing.
1. Enforce idempotent writes for safe retries.
1. Reject high-confidence secret-shaped text before persistence.

Coordination

1. Create project-scoped tasks.
1. Track task status and priority.
1. Enforce a permitted status-transition graph.
1. Protect task edits with optimistic versions.
1. Store task evidence references and linked memory IDs.
1. Create immutable agent handoffs.
1. Separate completed work, verified results, blockers, next steps, and evidence.
1. Require evidence for verified or partially verified handoffs.

Identity and access

- Create exactly one initial owner account.
- Hash passwords with asynchronous Node `scrypt`.
- Create device identities separately from sessions.
- Use opaque bearer sessions with hashed storage.
- Support device enrollment, role changes, revocation, and session expiry.
- Support project-level reader, editor, and owner permissions.
- Keep global owner access separate from project membership.
- Record security outcomes in an append-only audit table.

Desktop

- Start the backend automatically.
- Keep the backend bound to `127.0.0.1`.
- Require a private per-launch startup-token readiness handshake.
- Refuse to adopt an unrelated service occupying the configured port.
- Store production data under Electron `userData`, outside installed binaries.
- Preserve application data during uninstall and update.
- Create the first owner directly in the desktop UI without a setup code.
- Use a narrow, context-isolated IPC bridge only for first-owner bootstrap.

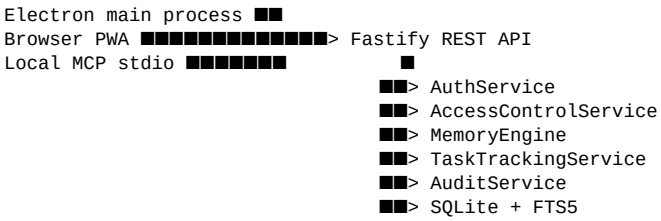
Repository map

```
D:\aimemory\
■■■ README.md                       Main quick-start guide
■■■ package.json                    Scripts, dependencies, Electron Builder config
■■■ package-lock.json               Locked dependency graph
■■■ tsconfig.json                   TypeScript compiler configuration
■■■ vitest.config.ts                Test configuration
■■■ public/                         Private authenticated PWA shell
■■■ site/                           Public marketing website source
■■■ src/
■   ■■■ api/                        Fastify REST API and static asset routes
■   ■■■ database/                   SQLite connection and migrations
■   ■■■ desktop/                    Backend worker, runtime paths, diagnostics, lifecycle
■   ■■■ electron/                   Electron main process and preload bridge
■   ■■■ memory/                     Memory domain engine and validation
■   ■■■ mcp/                        Local MCP server and schemas
■   ■■■ security/                   Auth, tokens, passwords, access control, audit
■   ■■■ tasks/                      Tasks, transitions, and immutable handoffs
■■■ scripts/
■   ■■■ demo.ts                     Persistence demonstration
■   ■■■ mcp-stdio.ts                MCP stdio entry point
■   ■■■ desktop-smoke.ts            Windows-only packaged smoke test
■   ■■■ create-setup-code.ts        Legacy controlled recovery helper
■   ■■■ preview-site.ts              Local marketing-site preview server
■   ■■■ build-site.ts               Static marketing-site build and validation
■■■ tests/                          API, desktop, MCP, PWA, memory, task, and site tests
■■■ docs/                           Detailed subsystem documentation
■■■ installer/                      Locally generated installer output, ignored by Git
```

The `installer/` directory is intentionally ignored because installers and block maps are generated artifacts. The current installer is documented in Current deliverables.

Architecture

Shared service graph



The Electron backend worker is a separately managed child process. This prevents synchronous `node:sqlite` work from blocking the Electron UI event loop.

Runtime boundaries

1. **Database:** one authoritative SQLite database per installation/configuration.
1. **REST:** `/api/v1`, loopback-only, authenticated except for narrowly defined public setup/status,

health, login, recovery, and desktop bootstrap boundaries.

1. **PWA:** static assets served from an explicit allowlist by Fastify.
1. **MCP:** stdio only; no HTTP transport or remote listener.
1. **Electron:** owns the backend child, supplies environment configuration, and supervises readiness.
1. **Website:** separate static files in `site/`; no access to private app data or APIs.

Phase history

Phase	Delivered capability
2	SQLite memory engine, FTS5, provenance, revisions, activity, migrations
3	Loopback REST API, authentication, project permissions, validation, rate limits, audits
4	Local stdio MCP server reusing the same services
5	Tasks, status transitions, optimistic versions, immutable handoffs
6	Responsive PWA and static-shell service worker
7	Windows Electron shell, packaged backend, data-path protection, desktop onboarding

Memory model

Record types

Memory records use one of:

1. `profile`
1. `project_fact`
1. `finding`
1. `constraint`
1. `requirement`
1. `note`

Provenance

Each memory revision has provenance fields:

1. `sourceKind`: `user_provided`, `file_observed`, `system_observed`, `agent_inferred`, or `test_result`.
1. Optional source reference.
1. Verification status: `unverified`, `user_confirmed`, `test_verified`, `superseded`, or `disputed`.
1. Optional verification method.
1. Optional observed timestamp.

Revisions and concurrency

The current memory projection is separate from immutable revisions. Updates require `expectedVersion`; a stale version is rejected rather than silently overwritten.

Memory creation and update transactions can reserve an idempotency key. Repeating the same request returns the original result; reusing the key with different input returns a conflict.

Search

SQLite FTS5 indexes the current memory projection. Historical revisions remain available through the history route but are not treated as current search results.

Projects, tasks, and handoffs

Projects

Projects are the primary authorization boundary for ordinary devices. The owner device can access all projects. Other devices need a project permission.

Task statuses

```
todo -> in_progress | blocked | cancelled
in_progress -> todo | blocked | completed | cancelled
blocked -> in_progress | cancelled
completed -> terminal
cancelled -> terminal
```

Every accepted transition records a status-history row with the actor, note, source status, destination status, and resulting version.

Task priorities

`low`, `normal`, `high`, and `urgent` are supported.

Handoffs

Handoffs are immutable. A handoff may link a task, destination device, memories, and other tasks. It stores:

1. Summary.
1. Completed work.
1. Verified results.
1. Blockers.
1. Next steps.
1. Evidence references.
1. Verification status.

Verified and partially verified handoffs require supporting results or evidence references.

Authentication and authorization

Passwords

Passwords are never stored in plaintext. The implementation uses asynchronous Node `scrypt` with:

1. Derived-key length: 64 bytes.
1. Random 16-byte salt.
1. Default cost parameters encoded with the stored hash.
1. Timing-safe verification.

The API requires at least 12 characters for owner creation and password changes.

Sessions

Sessions use random opaque bearer tokens. The raw token is returned only when a session is created. Only a SHA-256 hash is stored in `sessions`.

Each session has:

1. Absolute expiry.
1. Idle expiry.
1. Last-seen time.
1. Revocation state.
1. Device association.

The browser client stores the raw token only in JavaScript memory. Logout and protected-route 401 responses clear it.

Devices and roles

Global device roles:

1. `owner`: initial owner device with global project access.
1. `admin`: device-management capability; content still requires project membership.
1. `member`: project-scoped access only.

Project permission roles:

1. `reader`: read/search/history.
1. `editor`: reader plus memory/activity/task/handoff writes.
1. `owner`: editor plus project membership management.

Revoking a device revokes its sessions. Changing the owner device role is prohibited.

Audit logging

Security events are append-only and include identifiers, outcomes, request IDs, and safe metadata. Passwords, bearer tokens, setup codes, memory bodies, and internal exception details are not logged.

First-run desktop onboarding

Normal flow

For a new installation with no row in `owner_accounts`:

1. Electron starts the owned backend worker.
2. The backend completes the private startup-token health handshake.
3. The PWA calls `GET /api/v1/auth/setup/status`.
4. The UI renders **Create your OmniMemory account**.
5. The user enters a password, confirmation, device name, and device type.

6 The renderer invokes the narrow `omnimemory:create-owner` preload channel.

7 Electron main verifies the renderer origin and submits the request with the startup token.

8 `POST /api/v1/auth/setup/bootstrap` validates the request and opens a transaction.

9 The transaction creates the owner account, owner device, session, and audit event.

10 The session is returned to the renderer and held in memory.

11 The UI loads projects and navigates to the dashboard.

Security properties

1 The bootstrap route requires the per-launch startup token.

1 The token is created in Electron main and is not exposed through preload.

1 A normal browser cannot call the bootstrap route without the token.

1 The operation is allowed only while no owner exists.

1 Concurrent creation attempts cannot replace the owner.

1 Existing setup-code rows are not deleted, reset, or consumed by bootstrap.

Legacy recovery

`npm run setup-code` and `POST /api/v1/auth/setup` remain for controlled recovery/development. This path is not used by the normal desktop UI. A legacy setup-code row is retained for database compatibility and is not required for ordinary first-run onboarding.

REST API

The API base path is `/api/v1`. Responses use:

```
{
  "data": {},
  "meta": { "requestId": "..." }
}
```

Errors use:

```
{
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required",
    "requestId": "..."
  }
}
```

Authentication routes

```
GET  /api/v1/auth/setup/status
POST /api/v1/auth/setup                 Legacy recovery/development setup-code path
POST /api/v1/auth/setup/bootstrap       Electron-only first-owner path
POST /api/v1/auth/login
POST /api/v1/auth/password
POST /api/v1/auth/logout
GET  /api/v1/auth/me
```

```
POST /api/v1/auth/enrollments
POST /api/v1/auth/enrollments/claim
```

Device and permission routes

```
GET    /api/v1/devices
POST   /api/v1/devices/:id/revoke
PATCH  /api/v1/devices/:id/role
PUT    /api/v1/devices/:id/projects/:projectId
DELETE /api/v1/devices/:id/projects/:projectId
```

Memory and project routes

```
GET   /api/v1/projects
POST  /api/v1/projects
GET   /api/v1/projects/:projectId
GET   /api/v1/projects/:projectId/context
GET   /api/v1/projects/:projectId/memories/search
GET   /api/v1/projects/:projectId/memories/recent
POST  /api/v1/projects/:projectId/memories
GET   /api/v1/projects/:projectId/memories/:memoryId
PATCH /api/v1/projects/:projectId/memories/:memoryId
GET   /api/v1/projects/:projectId/memories/:memoryId/history
GET   /api/v1/projects/:projectId/activity
POST  /api/v1/projects/:projectId/activity
```

Task and handoff routes

```
POST  /api/v1/projects/:projectId/tasks
GET   /api/v1/projects/:projectId/tasks
GET   /api/v1/projects/:projectId/tasks/:taskId
PATCH /api/v1/projects/:projectId/tasks/:taskId
POST  /api/v1/projects/:projectId/tasks/:taskId/transitions
POST  /api/v1/projects/:projectId/handoffs
GET   /api/v1/projects/:projectId/handoffs
GET   /api/v1/projects/:projectId/handoffs/:handoffId
```

All public request bodies, query parameters, route IDs, and idempotency headers are bounded and validated. Cross-project resource lookups use generic not-found behavior to avoid leaking resource existence.

Browser PWA

The PWA is served by the same Fastify process and exposes only this static allowlist:

```
/
/app.js
/api-client.js
/styles.css
/manifest.webmanifest
/sw.js
/icon.svg
```

The service worker caches static assets only and bypasses `/api/` requests. It does not cache bearer tokens, private API responses, or offline writes.

The browser supports normal login and workspace use. First-owner creation is intended for the Electron desktop shell because the bootstrap operation is protected by the private startup token.

Local MCP integration

The MCP server is local stdio only:

```
$env:OMNIMEMORY_DB_PATH = '.\data\omnimemory.sqlite'
$env:OMNIMEMORY_MCP_TOKEN = '<local-session-token>'
npm run mcp
```

The token must be an existing OmniMemory bearer session token. It is authenticated again for every tool call, so expiry and revocation remain effective.

Available tools include:

```
search_memory, get_memory, get_project_context, list_recent_memories,
record_activity, create_memory, update_memory, get_memory_history,
create_task, list_tasks, get_task, update_task, transition_task,
create_handoff, list_handoffs, get_handoff
```

Every tool requires a project ID. Tool arguments are schema-validated and bounded. Arbitrary SQL, shell execution, unrestricted filesystem access, HTTP transport, and remote MCP are not provided.

SQLite database and migrations

SQLite is the authoritative local store. The application enables:

1. Foreign keys.
1. WAL journal mode.
1. A 5-second busy timeout.
1. Transactional migrations.

Migration versions:

Version	Name	Main contents
1	`initial_memory_engine`	Projects, memories, provenance, revisions, FTS5, activity, change log
2	`local_auth_devices_permissions_audit`	Owner, setup codes, devices, sessions, enrollments, permissions, audit
3	`write_idempotency_keys`	Persistent retry protection
4	`project_tasks_and_immutable_handoffs`	Tasks, status history, immutable handoffs

Important tables include:

1. `projects`
1. `memories`
1. `memory_revisions`
1. `provenance`
1. `memory_fts`

```
activity_events
change_log
owner_accounts
setup_codes
devices
sessions
device_enrollments
project_permissions
agent_activity_events
security_audit_events
idempotency_keys
tasks
task_status_history
handoffs
```

The application does not copy, replace, relocate, or delete a user database during startup, update, or uninstall. SQLite WAL and shared-memory sidecars live beside the database.

Configuration

Configuration is loaded from environment variables in `src/config.ts`.

Variable	Default	Purpose
`OMNIMEMORY_DB_PATH`	`./data/omnimemory.sqlite` in server mode; Electron `userData` in desktop mode	SQLite path
`OMNIMEMORY_PUBLIC_DIRECTORY`	`./public`	Static PWA directory
`OMNIMEMORY_STARTUP_TOKEN`	unset outside desktop	Private desktop readiness/bootstrap capability
`OMNIMEMORY_LOG_LEVEL`	`info`	`debug`, `info`, `warn`, or `error`
`OMNIMEMORY_HOST`	`127.0.0.1`	Loopback host; non-loopback values are rejected
`OMNIMEMORY_PORT`	`3210`	Local API port
`OMNIMEMORY_REQUEST_BODY_LIMIT`	`262144`	Request body bytes
`OMNIMEMORY_SESSION_TTL_MS`	30 days	Absolute session lifetime
`OMNIMEMORY_IDLE_SESSION_TTL_MS`	24 hours	Idle session lifetime
`OMNIMEMORY_ENROLLMENT_TTL_MS`	10 minutes	Device enrollment lifetime
`OMNIMEMORY_SETUP_CODE_TTL_MS`	10 minutes	Legacy recovery setup-code lifetime
`OMNIMEMORY_AUTH_RATE_LIMIT_MAX`	5	Auth attempts per rate-limit window
`OMNIMEMORY_MCP_TOKEN`	unset	Local MCP bearer token

The desktop runtime overrides host, port, public directory, database path, and startup token through the owned backend environment while preserving an explicit `OMNIMEMORY_DB_PATH` override.

Desktop runtime and packaging

Development

```
npm run desktop:dev
```

This builds TypeScript and launches Electron from the repository.

Packaging

```
npm run desktop:build:dir
npm run desktop:build
```

Electron Builder targets Windows x64 and NSIS. Electron 45.0.0 is pinned because its embedded Node runtime is compatible with the repository's `node:sqlite` requirement.

The packaged worker is launched from:

```
resources\app.asar.unpacked\dist\src\desktop\backend-worker.js
```

The archive path `app.asar` is not used as the child process working directory. The Electron `RunAsNode` fuse is enabled so the packaged Electron executable can run the backend worker with `ELECTRON_RUN_AS_NODE=1`.

Runtime checks

Before starting, the desktop process checks:

1. Bundled executable exists.
1. Backend worker exists.
1. User-data directory can be created.
1. Database parent directory is writable.

Startup readiness requires a successful health response with the per-launch startup token. The desktop does not adopt an unrelated process already using the port.

Diagnostics

The desktop writes safe diagnostics to:

```
%APPDATA%\OmniMemory\logs\desktop.log
```

Diagnostics contain lifecycle categories, status codes, exit codes, and signals. Startup tokens, bearer tokens, long hexadecimal secrets, passwords, and request bodies are not written.

Public marketing website

The public website is isolated under `site/` and builds to `dist-site/`. It is not the private PWA, does not access the database, and does not expose internal APIs or credentials.

Preview:

```
npm run site:preview
```

Build:

```
npm run site:build
```

The site build validates required assets, scans for development paths and secrets, and produces a static bundle suitable for GitHub Pages, Cloudflare Pages, Netlify, Vercel, or another static host.

Security model

Controls

1. Loopback-only host enforcement.
1. Foreign browser-origin rejection.
1. Request body limits.
1. Global and authentication-specific rate limits.
1. Zod validation at API boundaries.
1. Async `scrypt` password hashing.
1. Opaque token hashing.
1. Session expiry and revocation.
1. Device and project authorization.
1. Optimistic concurrency for memories and tasks.
1. Idempotency keys for retry-safe writes.
1. Secret-shaped text rejection.
1. Append-only audit records.
1. Safe public error mapping.
1. Electron context isolation, sandboxing, restricted navigation, and denied webviews.
1. Narrow preload bridge with no filesystem or shell access.
1. No remote listener, cloud identity, LAN access, or remote MCP.

Residual risks

OmniMemory does not provide OS-level isolation from another process running as the same local user. An administrator or same-user process can potentially inspect or interfere with local files, loopback traffic, or running processes. The installer is unsigned until a release-signing process is added. Backup, rollback, and password recovery remain operational responsibilities.

Installation and operating procedures

Development setup

```
npm install
npm run typecheck
npm run lint
npm run format:check
npm test
npm run build
```

Start the API

```
npm start
```

Default address:

```
http://127.0.0.1:3210/
```

First desktop launch

1. Install the Windows x64 NSIS installer.
2. Launch OmniMemory from the Start menu or desktop shortcut.
3. Complete **Create your OmniMemory account**.
4. Use a password of at least 12 characters and confirm it.
5. Choose the initial device name and type.
6. Create the account and confirm the dashboard loads.

No terminal, PowerShell, Node.js installation, setup-code copy/paste, or manual backend start is required for the installed application.

Existing installation

If an owner exists, the UI shows normal sign-in. The application does not recreate the owner, reset credentials, or create an empty replacement database.

Database backup

Stop OmniMemory before backing up the database and its SQLite sidecars. Do not copy a live WAL database while it is being written. Test a backup before upgrading.

Testing and verification

Primary checks:

```
npm run typecheck
npm run lint
npm run format:check
npm test
```

```
npm run build
npm run demo
```

The test suite covers:

1. API authentication and authorization.
1. First-owner desktop bootstrap.
1. Concurrent owner creation.
1. Existing-owner preservation.
1. Legacy setup-code state preservation.
1. Session expiry and revocation.
1. Memory revisions, provenance, search, and idempotency.
1. Task versions, transitions, and handoffs.
1. MCP discovery and tool calls.
1. PWA static assets and client error handling.
1. Desktop paths, startup handshakes, writable directories, and diagnostics.
1. Public website assets and security scans.

The current Linux environment can run automated checks and cross-build the Windows installer. It cannot claim real Windows GUI validation. `npm run desktop:smoke` reports the Windows-only limitation when run on Linux.

Current deliverables

The current rebuilt installer is staged at:

```
D:\aimemory\installer\OmniMemory-0.1.0-x64-Setup.exe
```

Current artifact metadata:

1. Size: 116,173,520 bytes.
1. SHA-256: `592c5d38b53faa00036a356be18b8374648499e09f3bfd9024d70d02bb93b475`.
1. Companion files: `.blockmap`, `latest.yml`, and `builder-debug.yml`.
1. ASAR includes the Electron main process, preload bridge, backend worker, MCP entry point, updated

PWA, and API client.

1. Electron fuse inspection reports `RunAsNode is Enabled`.

The artifact is cross-built and inspected. It still requires installation and launch on supported Windows for final GUI acceptance.

Limitations and deferred work

1. Windows-only packaged launch validation is still required.

1. The installer is unsigned.
1. No LAN, public networking, cloud sync, remote relay, or remote MCP.
1. No mobile client.
1. No embeddings or external AI API calls.
1. No automatic database backup, rollback, or cross-version database copying.
1. No password reset or password recovery flow beyond controlled legacy setup procedures.
1. No general offline write queue or automatic conflict merging.
1. The secret detector is heuristic.
1. Same-user local processes are not isolated at the OS level.

Maintainer checklist

Before changing behavior:

1. Read the relevant source file and all callers.
2. Inspect the existing tests and documentation.
3. Use an isolated temporary database for tests.
4. Preserve loopback-only binding and data-path rules.
5. Never log secrets, passwords, bearer tokens, or private memory content.
6. Add a regression test for non-trivial behavior.

Before committing:

```
git status --short --untracked-files=all
npm run typecheck
npm run lint
npm run format:check
npm test
npm run build
npm run demo
```

Before releasing the desktop package:

1. Build the Windows x64 NSIS installer.
2. Inspect the ASAR contents and Electron fuses.
3. Confirm the installer path and checksum.
4. Run `npm run desktop:smoke` on Windows.
5. Install and launch outside the repository.
6. Complete first-owner creation and verify the dashboard.
7. Verify an existing database is preserved across restart and update.
8. Publish only after physical Windows validation.

Related documentation

1. `README.md` — concise quick-start guide.
1. `docs/REQUIREMENTS.md` — functional and security requirements.
1. `docs/ARCHITECTURE.md` — phase architecture and service boundaries.
1. `docs/API_AND_MCP.md` — REST, PWA, MCP, and desktop contracts.
1. `docs/DATABASE_DESIGN.md` — migrations, tables, and persistence rules.
1. `docs/DESKTOP.md` — Electron runtime, installer, paths, and smoke testing.
1. `docs/MCP.md` — local MCP setup and agent workflow.
1. `docs/THREAT_MODEL.md` — threats, controls, and residual risk.
1. `docs/WEBSITE.md` — public website build and release workflow.